Plant Disease Diagnosis Agent — Backend Implementation Plan

For your mock UI, I recommend building a **vision-based plant disease diagnosis pipeline using Python, FastAPI, LangChain, and LangGraph**. The backend should return structured data that directly populates your existing UI — disease name, symptoms, diagnostic confidence, warnings, and recommended next actions.

The key design decision: **don't build a general-purpose autonomous agent. Build a controlled diagnostic workflow with two specialized prompts and deterministic validation steps.** This is easier to test, cheaper to operate, and safer for agricultural recommendations.

1. Recommended architecture

1. Image and crop metadata

Leaf photo, crop type, location (optional), symptoms and growing conditions

↓

2. FastAPI validation layer

File validation · Image quality · EXIF removal · Size limits

↓

3. Disease identification — Prompt 1

A vision-capable model examines the leaf, recognizes visual signs, compares plausible diseases, and returns a structured differential diagnosis.

↓

4. Evidence and safety validation

Check crop/disease compatibility, retrieve trusted agricultural references, and detect insufficient evidence.

↓

5. Diagnosis and action plan — Prompt 2

Converts validated findings into UI-ready explanations, signs that fit, immediate actions, monitoring guidance, and treatment precautions.

↓

6. Validated JSON response

Pydantic schema → API response → your existing UI

FastAPI's `UploadFile` supports file uploads without necessarily loading every upload fully into memory. LangChain supports structured model output using Pydantic schemas, which is ideal for this UI-driven application.

FastAPI +1

2. Technology stack

Component	Recommendation	Responsibility
API	FastAPI	Upload and diagnosis endpoints
Agent orchestration	LangGraph	Manage the diagnostic workflow and branching
LLM integration	LangChain	Model invocation, prompts and structured output
Vision model	Vision-capable multimodal LLM	Interpret leaf photographs
Output validation	Pydantic v2	Enforce response contracts
Knowledge retrieval	Qdrant + embeddings, when needed	Retrieve trusted disease and treatment guidance
Database	PostgreSQL	Diagnosis history, feedback and metadata
Image storage	S3-compatible object storage	Store uploaded images if retention is required
Background jobs	Add Celery or a similar worker only when needed	Handle slow or asynchronous processing

For your cost-conscious implementation, start with a hosted vision model for accuracy and rapid development, then benchmark a local vision model through Ollama if inference costs justify the trade-off. A text-only local model cannot directly interpret a leaf photograph without an additional image-analysis component.

Important: LangGraph should orchestrate a small workflow, not give the LLM unrestricted control over tools. Initially, five or six explicit steps are enough.

3. Prompt 1 — Disease identification

This is the core vision prompt. Its job is to identify plausible diseases, not prescribe treatment.

Use it with a multimodal model that receives the actual image bytes or image URL through its supported vision interface.

Suggested file: `app/prompts/disease_identification.py`

Python

```
- Shape, color, size and distribution of lesions
```

Why this prompt works

- It avoids forcing the model to return a disease even when the evidence is weak.
- It considers non-disease causes, such as nutrient deficiencies.
- It separates observations from hypotheses.
- It provides evidence that the next stage can use rather than asking the second prompt to diagnose everything again.

Your sample UI shows tomato early blight. The model should be able to return early blight as a *possible* disease when the characteristic lesions are visible, while preserving alternatives when the image cannot distinguish them.

4. Prompt 2 — Diagnosis explanation and recommended actions

This prompt receives the first prompt's structured findings plus trusted agricultural reference material. It prepares the content displayed in your UI.

Suggested file: `app/prompts/diagnosis_response.py`

Python

```
reference material.

1. DIAGNOSIS
   Give a concise disease or condition name.
   Use "Possible disease" or "Possible cause" when unconfirmed.
```

One important implementation detail

Do not let Prompt 2 invent recommendations from its general knowledge when reliable reference material is missing. For an MVP, maintain a curated set of agricultural guidance documents, extension publications, and region-appropriate treatment references.

You do not need RAG to identify a disease from an image. RAG is useful for **grounding the explanation and treatment guidance** in authoritative sources.

5. Define a response contract that matches your UI

Don't let either prompt return arbitrary JSON. Define a single Pydantic schema and require the final response to conform to it. LangChain's structured-output capabilities support this pattern.

GitHub +1

Here is a proposed schema aligned with the mockup.

Python

```python
# app/schemas/diagnosis.py

from typing import Literal
from pydantic import BaseModel, Field


class Symptom(BaseModel):
    name: str
    evidence_source: Literal["image", "user_report", "reference"]
    explanation: str


class DiseaseCandidate(BaseModel):
    name: str
    category: Literal[
        "fungal",
        "bacterial",
        "viral",
        "pest"
```

Confidence is not the same as probability. Unless you validate a model against a representative, labeled dataset and calibrate its scores, expose qualitative confidence categories with explanations rather than fabricated percentages.

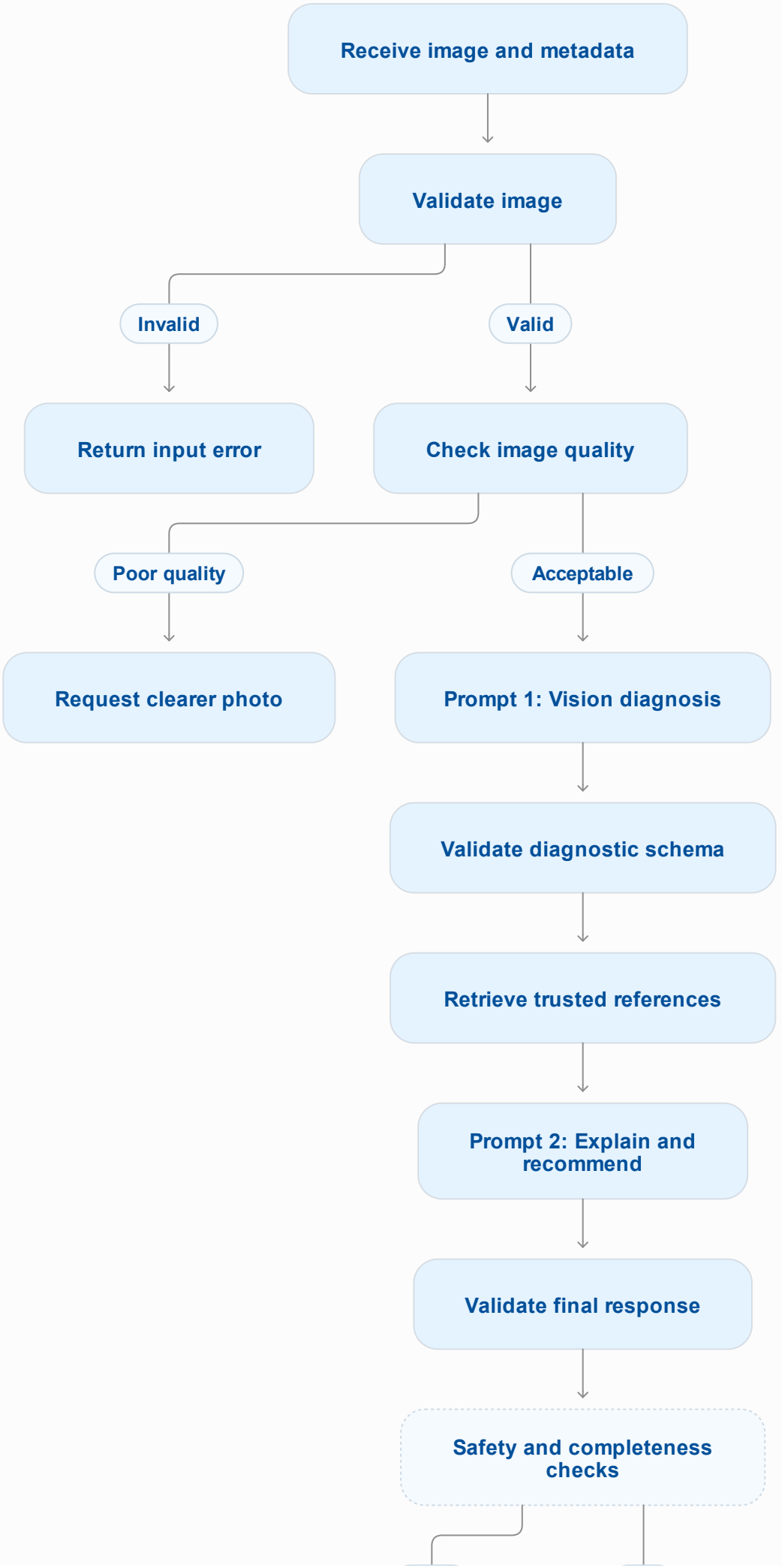
Map these fields to your existing UI:

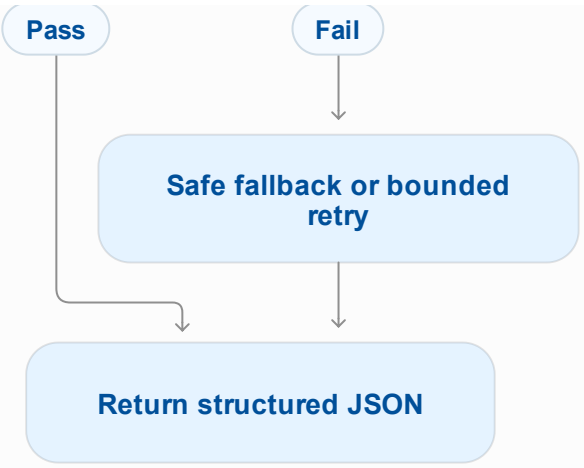
UI element	API response field
“Possible disease” badge	`status`
“Early blight” heading	`diagnosis_name`
Disease summary	`description`
“Signs that may fit”	`symptoms`
Alternative causes	`alternatives`
“Needs confirmation”	`expert_confirmation_required`
“What you can do now” steps	`actions`
Follow-up monitoring	`monitoring_guidance`
Treatment warning	`diagnostic_limitations` and action safety notes

Keep `category_label` and the UI's exact display wording in your backend presentation mapping, rather than asking the model to invent badges or UI labels.

6. The LangGraph prompt chain

This is the implementation I would build first. Use a graph with explicit nodes and conditional routing.


Receive image and metadata
Validate image
Invalid
Valid
Return input error
Check image quality
Poor quality
Acceptable
Request clearer photo
Prompt 1: Vision diagnosis
Validate diagnostic schema
Retrieve trusted references
Prompt 2: Explain and recommend
Validate final response
Safety and completeness checks



Recommended project structure

Plain text

```
plant-diagnosis/
├── app/
│   ├── main.py
│   ├── api/
│   │   └── diagnosis.py
│   ├── agents/
│   │   ├── graph.py
│   │   ├── state.py
│   │   └── nodes.py
│   ├── prompts/
│   │   ├── disease_identification.py
│   │   └── diagnosis_response.py
│   ├── schemas/
│   │   └── diagnosis.py
│   ├── services/
│   │   ├── vision.py
│   │   ├── image_quality.py
│   │   ├── knowledge_retrieval.py
│   │   └── safety.py
│   └── repositories/
```

Define the graph state

Python

```python
# app/agents/state.py

from typing import TypedDict, Any
```

```
class DiagnosisState(TypedDict, total=False):
    image_bytes: bytes
    crop_name: str
    plant_part: str
    environment: str
    region: str
    additional_symptoms: str

    image_quality: str
    diagnostic_findings: dict[str, Any]
    reference_context: list[dict[str, Any]]
    final_report: dict[str, Any]

    error: str
```

Implement the two model calls

The following example uses LangChain's `with_structured_output()` interface. `VisionModel` is a placeholder for the adapter you implement for your chosen multimodal provider; it must pass the image as actual image input, not as a textual filename.

Python

```
# app/agents/nodes.py

from langchain_core.prompts import ChatPromptTemplate

from app.schemas.diagnosis import DiagnosisResponse
from app.prompts.disease_identification import (
    DISEASE_IDENTIFICATION_PROMPT,
)
from app.prompts.diagnosis_response import (
    DIAGNOSIS_RESPONSE_PROMPT,
)


# Configure this adapter for your chosen vision-capable model.
# Its implementation must support image + text input.
vision_llm = VisionModel(temperature=0)

# The final report is a Pydantic-validated object.
report_llm = text_llm.with_structured_output(DiagnosisResponse)
```

`VisionModel`, `text_llm`, `invoke_vision`, and the image-quality and retrieval functions are integration points, not built-in LangChain APIs. Implement them for the provider you select. You should

also define a separate Pydantic schema for `diagnostic_findings`, rather than leaving the first model's output as an unconstrained dictionary.

Wire the graph

Python

```python
# app/agents/graph.py

from langgraph.graph import StateGraph, START, END
from app.agents.state import DiagnosisState
from app.agents.nodes import identify_disease, generate_report


def route_quality(state: DiagnosisState) -> str:
    if state.get("image_quality") != "acceptable":
        return "stop"
    return "diagnose"


def build_graph():
    graph = StateGraph(DiagnosisState)

    graph.add_node("check_quality", check_image_quality)
    graph.add_node("identify_disease", identify_disease)
    graph.add_node("retrieve_references", retrieve_references)
    graph.add_node("generate_report", generate_report)
```

The quality-check and validation nodes should return a clear, safe response when they cannot proceed. In production, I'd make the graph's routing explicit for errors and inconclusive diagnoses rather than treating every failure as a successful report.

7. Expose the agent through FastAPI

Keep the API thin. FastAPI should validate the upload, invoke the graph, and return the validated report. It should not contain the diagnostic logic itself.

Python

```python
# app/api/diagnosis.py

from typing import Annotated

from fastapi import APIRouter, File, Form, HTTPException, UploadFile
```

```python
from app.agents.graph import diagnosis_graph
from app.schemas.diagnosis import DiagnosisResponse

router = APIRouter(prefix="/api/v1", tags=["diagnosis"])

MAX_IMAGE_BYTES = 10 * 1024 * 1024

ALLOWED_TYPES = {
    "image/jpeg",
    "image/png",
    "image/webp",
}
```

This is a starter endpoint; wire `check_image_quality` and the other graph nodes before deploying. Also enforce upload limits at the reverse proxy, verify decoded image dimensions, strip unnecessary metadata, and configure timeouts and concurrency limits.

One useful API design decision: return a normal JSON report for an inconclusive diagnosis, but use HTTP errors for invalid requests and service failures. That way, your UI can display a meaningful "Please upload a clearer image" state without treating it as an application error.

8. Knowledge retrieval: what should go into RAG?

I would not index the entire internet. Start with a small, curated knowledge base.

Disease reference documents

Disease descriptions, characteristic symptoms, affected crops, lookalike conditions, and diagnostic limitations.

Treatment and safety guidance

Official extension publications, locally approved treatments, label restrictions, protective equipment, and harvest intervals.

Regional applicability

Crop-specific recommendations relevant to the user's region, with source date and applicability metadata.

For each document, preserve metadata such as `crop`, `disease`, `region`, `publisher`, `source_url`, `published_at`, and `reviewed_at`. Retrieve by crop and relevant diagnostic candidates, then rerank the results for relevance.

For the first MVP, you can start with PostgreSQL and curated reference records. Introduce Qdrant when semantic retrieval across a larger document collection provides clear value.

9. Implementation roadmap

Phase 1 — Working diagnosis

First milestone

Implement upload validation, the vision model, Prompt 1, Pydantic schemas, and the `/diagnose` endpoint. Test with tomato leaves first.

Phase 2 — Diagnostic workflow

Add LangGraph routing, image-quality checks, Prompt 2, and explicit inconclusive-diagnosis handling.

Phase 3 — Grounded advice

Add curated agricultural references, retrieval, source metadata, and safety checks before presenting treatment guidance.

Phase 4 — Evaluation and deployment

Build a labeled test set, measure top-1/top-3 diagnostic accuracy, assess false confident diagnoses, test schema validity, and containerize the backend.

10. Production acceptance criteria

Before exposing diagnoses to farmers, test the following cases:

- Clear image with a characteristic disease.
- Blurry or distant leaf image.
- Healthy leaf with no visible disease.
- Two diseases with similar visual symptoms.
- Nutrient deficiency or pest damage resembling a disease.
- Crop name that conflicts with the image.
- A disease for which the knowledge base has no verified treatment guidance.
- Model timeout, invalid structured output, and retrieval failure.

Track disease-level precision and recall, top-3 accuracy, inconclusive-case handling, and the rate of unsupported treatment recommendations. Do not evaluate the system only by whether its JSON parses.

My final recommendation: Start with one multimodal diagnosis call, one grounded explanation call, and a small LangGraph workflow. Add RAG for trustworthy advice, not as a substitute for visual diagnosis. Avoid multiple autonomous agents until evaluation shows that a more complex architecture improves results.

For reference, the implementation patterns above follow the current LangChain structured-output documentation ↗ and FastAPI file-upload documentation ↗.